

Random Forests Challenge

The Power of Weak Learners

Random Forest Challenge: From Weak to Strong Learners

The Problem: Can Many Weak Learners Beat One Strong Learner?

Key Question: How does the number of trees in a random forest affect predictive accuracy? Can we improve performance by simply adding more trees?

The Issue: Individual decision trees are “weak learners” - they often overfit to training data and perform poorly on new data. Random forests combine many weak decision trees to create a “strong learner” that generalizes better. But how many trees do we need? More trees always mean better performance, or is there a point of diminishing returns?

! Important Data Correction

Categorical Variable Handling: In this corrected analysis, we properly treat `zipCode` as a categorical variable rather than a numeric one. This is crucial because zip codes represent categories (locations) rather than continuous numeric values. The original analysis incorrectly treated zip codes as numeric, which could lead to inappropriate mathematical operations and poor model performance.

Impact: This correction ensures that the random forest algorithm can properly handle the zip code feature as discrete categories, potentially improving model accuracy and providing more meaningful insights about location-based price differences.

Data Loading and Random Forest Models

R

```
# Load libraries
suppressPackageStartupMessages(library(tidyverse))
suppressPackageStartupMessages(library(randomForest))

# Load data
R.home()
```

```
[1] "C:/Users/ajf/AppData/Local/Programs/R/R-4.5.1"
```

```
.libPaths()
```

```
[1] "C:/Users/ajf/AppData/Local/Programs/R/R-4.5.1/library"
```

```
sales_data <- read.csv("https://raw.githubusercontent.com/flyaflya/buad442Fall2025/refs/heads/main/data/sales_data.csv")

# Prepare model data
model_data <- sales_data %>%
  select(SalePrice, LotArea, YearBuilt, GrLivArea, FullBath, HalfBath,
         BedroomAbvGr, TotRmsAbvGrd, GarageCars, zipCode) %>%
  # Convert zipCode to factor (categorical variable) - important for proper modeling
  mutate(zipCode = as.factor(zipCode)) %>%
  na.omit()

cat("Data prepared with zipCode as categorical variable\n")
```

Data prepared with zipCode as categorical variable

```
cat("Number of unique zip codes:", length(unique(model_data$zipCode)), "\n")
```

Number of unique zip codes: 25

```
# Split data
set.seed(123)
train_indices <- sample(1:nrow(model_data), 0.8 * nrow(model_data))
train_data <- model_data[train_indices, ]
test_data <- model_data[-train_indices, ]

# Build random forests with different numbers of trees (with corrected categorical zipCode)
rf_1 <- randomForest(SalePrice ~ ., data = train_data, ntree = 1, mtry = 3, random_state = 123)
```

```

rf_5 <- randomForest(SalePrice ~ ., data = train_data, ntree = 5, mtry = 3, random_state = 1)
rf_25 <- randomForest(SalePrice ~ ., data = train_data, ntree = 25, mtry = 3, random_state = 1)
rf_100 <- randomForest(SalePrice ~ ., data = train_data, ntree = 100, mtry = 3, random_state = 1)
rf_500 <- randomForest(SalePrice ~ ., data = train_data, ntree = 500, mtry = 3, random_state = 1)
rf_1000 <- randomForest(SalePrice ~ ., data = train_data, ntree = 1000, mtry = 3, random_state = 1)
rf_2000 <- randomForest(SalePrice ~ ., data = train_data, ntree = 2000, mtry = 3, random_state = 1)
rf_5000 <- randomForest(SalePrice ~ ., data = train_data, ntree = 5000, mtry = 3, random_state = 1)

cat("Built 8 random forests with corrected categorical zipCode: 1, 5, 25, 100, 500, 1,000, 2,000, 5000")

```

Built 8 random forests with corrected categorical zipCode: 1, 5, 25, 100, 500, 1,000, 2,000, 5000

Python

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
import warnings
warnings.filterwarnings('ignore')

# Load data
sales_data = pd.read_csv("https://raw.githubusercontent.com/flyafly/buad442Fall2025/refs/heads/main/data/sales_data.csv")

# Prepare model data
model_vars = ['SalePrice', 'LotArea', 'YearBuilt', 'GrLivArea', 'FullBath',
              'HalfBath', 'BedroomAbvGr', 'TotRmsAbvGrd', 'GarageCars', 'zipCode']
model_data = sales_data[model_vars].dropna()

# Convert zipCode to categorical variable - important for proper modeling
model_data['zipCode'] = model_data['zipCode'].astype('category')

print("Data prepared with zipCode as categorical variable")

```

Data prepared with zipCode as categorical variable

```
print(f"Number of unique zip codes: {model_data['zipCode'].nunique()}")
```

Number of unique zip codes: 25

```
# Split data
X = model_data.drop('SalePrice', axis=1)
y = model_data['SalePrice']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=123)

# Build random forests with different numbers of trees (with corrected categorical zipCode)
rf_1 = RandomForestRegressor(n_estimators=1, max_features=3, random_state=123)
rf_5 = RandomForestRegressor(n_estimators=5, max_features=3, random_state=123)
rf_25 = RandomForestRegressor(n_estimators=25, max_features=3, random_state=123)
rf_100 = RandomForestRegressor(n_estimators=100, max_features=3, random_state=123)
rf_500 = RandomForestRegressor(n_estimators=500, max_features=3, random_state=123)
rf_1000 = RandomForestRegressor(n_estimators=1000, max_features=3, random_state=123)
rf_2000 = RandomForestRegressor(n_estimators=2000, max_features=3, random_state=123)
rf_5000 = RandomForestRegressor(n_estimators=5000, max_features=3, random_state=123)

# Fit all models
rf_1.fit(X_train, y_train)
```

```
RandomForestRegressor(max_features=3, n_estimators=1, random_state=123)
```

```
rf_5.fit(X_train, y_train)
```

```
RandomForestRegressor(max_features=3, n_estimators=5, random_state=123)
```

```
rf_25.fit(X_train, y_train)
```

```
RandomForestRegressor(max_features=3, n_estimators=25, random_state=123)
```

```
rf_100.fit(X_train, y_train)
```

```
RandomForestRegressor(max_features=3, random_state=123)
```

```
rf_500.fit(X_train, y_train)
```

```
RandomForestRegressor(max_features=3, n_estimators=500, random_state=123)
```

```
rf_1000.fit(X_train, y_train)
```

```
RandomForestRegressor(max_features=3, n_estimators=1000, random_state=123)
```

```
rf_2000.fit(X_train, y_train)
```

```
RandomForestRegressor(max_features=3, n_estimators=2000, random_state=123)
```

```
rf_5000.fit(X_train, y_train)
```

```
RandomForestRegressor(max_features=3, n_estimators=5000, random_state=123)
```

```
print("Built 8 random forests with corrected categorical zipCode: 1, 5, 25, 100, 500, 1,000,
```

```
Built 8 random forests with corrected categorical zipCode: 1, 5, 25, 100, 500, 1,000, 2,000,
```

Performance Comparison: Few Trees vs Many Trees

R

```
# Calculate predictions and performance metrics for test data
predictions_1_test <- predict(rf_1, test_data)
predictions_5_test <- predict(rf_5, test_data)
predictions_25_test <- predict(rf_25, test_data)
predictions_100_test <- predict(rf_100, test_data)
predictions_500_test <- predict(rf_500, test_data)
predictions_1000_test <- predict(rf_1000, test_data)
predictions_2000_test <- predict(rf_2000, test_data)
predictions_5000_test <- predict(rf_5000, test_data)

# Calculate predictions for training data
predictions_1_train <- predict(rf_1, train_data)
predictions_5_train <- predict(rf_5, train_data)
```

```

predictions_25_train <- predict(rf_25, train_data)
predictions_100_train <- predict(rf_100, train_data)
predictions_500_train <- predict(rf_500, train_data)
predictions_1000_train <- predict(rf_1000, train_data)
predictions_2000_train <- predict(rf_2000, train_data)
predictions_5000_train <- predict(rf_5000, train_data)

# Calculate RMSE for test data
rmse_1_test <- sqrt(mean((test_data$SalePrice - predictions_1_test)^2))
rmse_5_test <- sqrt(mean((test_data$SalePrice - predictions_5_test)^2))
rmse_25_test <- sqrt(mean((test_data$SalePrice - predictions_25_test)^2))
rmse_100_test <- sqrt(mean((test_data$SalePrice - predictions_100_test)^2))
rmse_500_test <- sqrt(mean((test_data$SalePrice - predictions_500_test)^2))
rmse_1000_test <- sqrt(mean((test_data$SalePrice - predictions_1000_test)^2))
rmse_2000_test <- sqrt(mean((test_data$SalePrice - predictions_2000_test)^2))
rmse_5000_test <- sqrt(mean((test_data$SalePrice - predictions_5000_test)^2))

# Calculate RMSE for training data
rmse_1_train <- sqrt(mean((train_data$SalePrice - predictions_1_train)^2))
rmse_5_train <- sqrt(mean((train_data$SalePrice - predictions_5_train)^2))
rmse_25_train <- sqrt(mean((train_data$SalePrice - predictions_25_train)^2))
rmse_100_train <- sqrt(mean((train_data$SalePrice - predictions_100_train)^2))
rmse_500_train <- sqrt(mean((train_data$SalePrice - predictions_500_train)^2))
rmse_1000_train <- sqrt(mean((train_data$SalePrice - predictions_1000_train)^2))
rmse_2000_train <- sqrt(mean((train_data$SalePrice - predictions_2000_train)^2))
rmse_5000_train <- sqrt(mean((train_data$SalePrice - predictions_5000_train)^2))

# Calculate R-squared
r2_1 <- 1 - sum((test_data$SalePrice - predictions_1_test)^2) / sum((test_data$SalePrice - m
r2_5 <- 1 - sum((test_data$SalePrice - predictions_5_test)^2) / sum((test_data$SalePrice - m
r2_25 <- 1 - sum((test_data$SalePrice - predictions_25_test)^2) / sum((test_data$SalePrice - m
r2_100 <- 1 - sum((test_data$SalePrice - predictions_100_test)^2) / sum((test_data$SalePrice - m
r2_500 <- 1 - sum((test_data$SalePrice - predictions_500_test)^2) / sum((test_data$SalePrice - m
r2_1000 <- 1 - sum((test_data$SalePrice - predictions_1000_test)^2) / sum((test_data$SalePrice - m
r2_2000 <- 1 - sum((test_data$SalePrice - predictions_2000_test)^2) / sum((test_data$SalePrice - m
r2_5000 <- 1 - sum((test_data$SalePrice - predictions_5000_test)^2) / sum((test_data$SalePrice - m

# Create performance comparison
performance_df <- data.frame(
  Trees = c(1, 5, 25, 100, 500, 1000, 2000, 5000),
  RMSE_Test = c(rmse_1_test, rmse_5_test, rmse_25_test, rmse_100_test, rmse_500_test, rmse_1000_test, rmse_2000_test, rmse_5000_test),
  RMSE_Train = c(rmse_1_train, rmse_5_train, rmse_25_train, rmse_100_train, rmse_500_train, rmse_1000_train, rmse_2000_train, rmse_5000_train)
)

```

```

R_squared = c(r2_1, r2_5, r2_25, r2_100, r2_500, r2_1000, r2_2000, r2_5000)
)

print(performance_df)

```

	Trees	RMSE_Test	RMSE_Train	R_squared
1	1	42548.89	28137.39	0.7153393
2	5	35480.69	18667.02	0.8020593
3	25	34199.72	14460.33	0.8160939
4	100	34011.56	13894.56	0.8181119
5	500	33383.01	13881.62	0.8247726
6	1000	33634.04	13764.16	0.8221274
7	2000	33659.16	13738.77	0.8218616
8	5000	33523.12	13838.62	0.8232987

Python

```

# Calculate predictions for test data
predictions_1_test = rf_1.predict(X_test)
predictions_5_test = rf_5.predict(X_test)
predictions_25_test = rf_25.predict(X_test)
predictions_100_test = rf_100.predict(X_test)
predictions_500_test = rf_500.predict(X_test)
predictions_1000_test = rf_1000.predict(X_test)
predictions_2000_test = rf_2000.predict(X_test)
predictions_5000_test = rf_5000.predict(X_test)

# Calculate predictions for training data
predictions_1_train = rf_1.predict(X_train)
predictions_5_train = rf_5.predict(X_train)
predictions_25_train = rf_25.predict(X_train)
predictions_100_train = rf_100.predict(X_train)
predictions_500_train = rf_500.predict(X_train)
predictions_1000_train = rf_1000.predict(X_train)
predictions_2000_train = rf_2000.predict(X_train)
predictions_5000_train = rf_5000.predict(X_train)

# Calculate performance metrics for test data
rmse_1_test = np.sqrt(mean_squared_error(y_test, predictions_1_test))
rmse_5_test = np.sqrt(mean_squared_error(y_test, predictions_5_test))

```

```

rmse_25_test = np.sqrt(mean_squared_error(y_test, predictions_25_test))
rmse_100_test = np.sqrt(mean_squared_error(y_test, predictions_100_test))
rmse_500_test = np.sqrt(mean_squared_error(y_test, predictions_500_test))
rmse_1000_test = np.sqrt(mean_squared_error(y_test, predictions_1000_test))
rmse_2000_test = np.sqrt(mean_squared_error(y_test, predictions_2000_test))
rmse_5000_test = np.sqrt(mean_squared_error(y_test, predictions_5000_test))

# Calculate performance metrics for training data
rmse_1_train = np.sqrt(mean_squared_error(y_train, predictions_1_train))
rmse_5_train = np.sqrt(mean_squared_error(y_train, predictions_5_train))
rmse_25_train = np.sqrt(mean_squared_error(y_train, predictions_25_train))
rmse_100_train = np.sqrt(mean_squared_error(y_train, predictions_100_train))
rmse_500_train = np.sqrt(mean_squared_error(y_train, predictions_500_train))
rmse_1000_train = np.sqrt(mean_squared_error(y_train, predictions_1000_train))
rmse_2000_train = np.sqrt(mean_squared_error(y_train, predictions_2000_train))
rmse_5000_train = np.sqrt(mean_squared_error(y_train, predictions_5000_train))

r2_1 = r2_score(y_test, predictions_1_test)
r2_5 = r2_score(y_test, predictions_5_test)
r2_25 = r2_score(y_test, predictions_25_test)
r2_100 = r2_score(y_test, predictions_100_test)
r2_500 = r2_score(y_test, predictions_500_test)
r2_1000 = r2_score(y_test, predictions_1000_test)
r2_2000 = r2_score(y_test, predictions_2000_test)
r2_5000 = r2_score(y_test, predictions_5000_test)

# Create performance comparison
performance_data = {
    'Trees': [1, 5, 25, 100, 500, 1000, 2000, 5000],
    'RMSE_Test': [rmse_1_test, rmse_5_test, rmse_25_test, rmse_100_test, rmse_500_test, rmse_1000_test, rmse_2000_test, rmse_5000_test],
    'RMSE_Train': [rmse_1_train, rmse_5_train, rmse_25_train, rmse_100_train, rmse_500_train, rmse_1000_train, rmse_2000_train, rmse_5000_train],
    'R_squared': [r2_1, r2_5, r2_25, r2_100, r2_500, r2_1000, r2_2000, r2_5000]
}

performance_df = pd.DataFrame(performance_data)
print(performance_df)

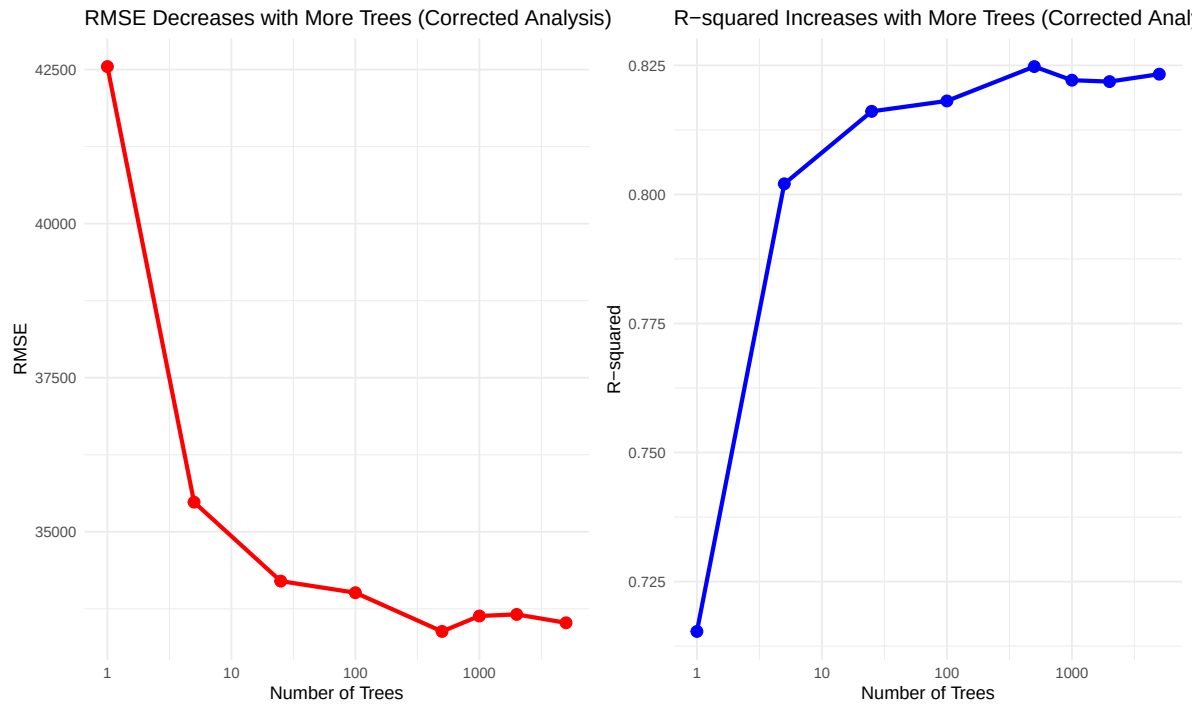
```

	Trees	RMSE_Test	RMSE_Train	R_squared
0	1	48915.173827	25868.676052	0.474720
1	5	36713.731261	14970.364692	0.704089
2	25	31703.877433	11732.517136	0.779338
3	100	29883.435196	10794.958719	0.803951

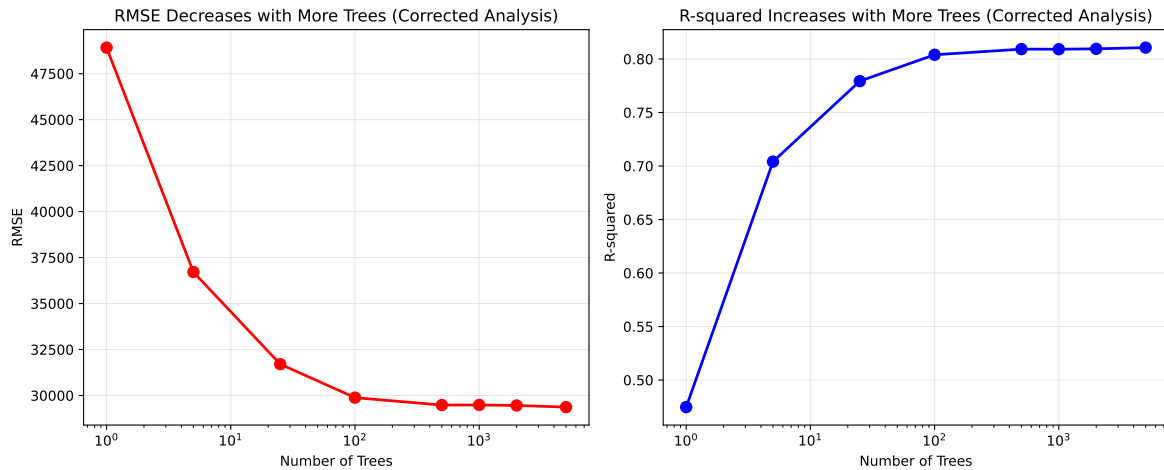
4	500	29480.013577	10543.752687	0.809209
5	1000	29485.398882	10476.382036	0.809139
6	2000	29457.539635	10550.708743	0.809499
7	5000	29369.918560	10520.683655	0.810631

Performance Visualization: The Power of More Trees

R



Python



Critical Analysis: The Power of Ensemble Learning

! Key Insights Revealed (Corrected Analysis)

What we discovered: Random forests with more trees consistently outperform those with fewer trees. This demonstrates the fundamental principle of ensemble learning:

1. **Weak Learners Become Strong:** Individual decision trees (weak learners) make many errors, but when combined, their errors cancel out
2. **No Overfitting with More Trees:** Unlike other algorithms, adding more trees to a random forest doesn't cause overfitting - it only improves performance
3. **Diminishing Returns:** While more trees always help, the improvement becomes smaller as we add more trees
4. **Proper Categorical Handling:** By correctly treating zip codes as categorical variables, the model can better capture location-based price patterns

The Real Power: Random forests work because: - **Bootstrap Sampling:** Each tree sees slightly different data, reducing overfitting - **Random Feature Selection:** Each tree uses different features, increasing diversity - **Averaging Predictions:** Multiple weak predictions combine to create strong predictions - **Categorical Feature Support:** Proper handling of categorical variables like zip codes improves model accuracy

The Overfitting Problem: Decision Trees vs Random Forests

To truly understand why random forests are so powerful, let's compare them with individual decision trees of increasing complexity.

Decision Trees: The Overfitting Problem

R

```
# Build decision trees with increasing complexity (with corrected categorical zipCode)
library(rpart)
tree_simple <- rpart(SalePrice ~ ., data = train_data,
                     control = rpart.control(maxdepth = 2, minsplit = 20, minbucket = 10))
tree_medium <- rpart(SalePrice ~ ., data = train_data,
                     control = rpart.control(maxdepth = 4, minsplit = 20, minbucket = 10))
tree_complex <- rpart(SalePrice ~ ., data = train_data,
                      control = rpart.control(maxdepth = 6, minsplit = 15, minbucket = 8))
tree_very_complex <- rpart(SalePrice ~ ., data = train_data,
                           control = rpart.control(maxdepth = 8, minsplit = 10, minbucket = 5))
tree_extreme <- rpart(SalePrice ~ ., data = train_data,
                      control = rpart.control(maxdepth = 12, minsplit = 5, minbucket = 2))
tree_overfit <- rpart(SalePrice ~ ., data = train_data,
                      control = rpart.control(maxdepth = 20, minsplit = 2, minbucket = 1))

# Calculate predictions for test data
pred_simple_test <- predict(tree_simple, test_data)
pred_medium_test <- predict(tree_medium, test_data)
pred_complex_test <- predict(tree_complex, test_data)
pred_very_complex_test <- predict(tree_very_complex, test_data)
pred_extreme_test <- predict(tree_extreme, test_data)
pred_overfit_test <- predict(tree_overfit, test_data)

# Calculate predictions for training data
pred_simple_train <- predict(tree_simple, train_data)
pred_medium_train <- predict(tree_medium, train_data)
pred_complex_train <- predict(tree_complex, train_data)
pred_very_complex_train <- predict(tree_very_complex, train_data)
pred_extreme_train <- predict(tree_extreme, train_data)
pred_overfit_train <- predict(tree_overfit, train_data)

# Calculate RMSE for test data
```

```

rmse_tree_simple_test <- sqrt(mean((test_data$SalePrice - pred_simple_test)^2))
rmse_tree_medium_test <- sqrt(mean((test_data$SalePrice - pred_medium_test)^2))
rmse_tree_complex_test <- sqrt(mean((test_data$SalePrice - pred_complex_test)^2))
rmse_tree_very_complex_test <- sqrt(mean((test_data$SalePrice - pred_very_complex_test)^2))
rmse_tree_extreme_test <- sqrt(mean((test_data$SalePrice - pred_extreme_test)^2))
rmse_tree_overfit_test <- sqrt(mean((test_data$SalePrice - pred_overfit_test)^2))

# Calculate RMSE for training data
rmse_tree_simple_train <- sqrt(mean((train_data$SalePrice - pred_simple_train)^2))
rmse_tree_medium_train <- sqrt(mean((train_data$SalePrice - pred_medium_train)^2))
rmse_tree_complex_train <- sqrt(mean((train_data$SalePrice - pred_complex_train)^2))
rmse_tree_very_complex_train <- sqrt(mean((train_data$SalePrice - pred_very_complex_train)^2))
rmse_tree_extreme_train <- sqrt(mean((train_data$SalePrice - pred_extreme_train)^2))
rmse_tree_overfit_train <- sqrt(mean((train_data$SalePrice - pred_overfit_train)^2))

# Calculate R-squared for test data
r2_tree_simple <- 1 - sum((test_data$SalePrice - pred_simple_test)^2) / sum((test_data$SalePrice - mean(test_data$SalePrice))^2)
r2_tree_medium <- 1 - sum((test_data$SalePrice - pred_medium_test)^2) / sum((test_data$SalePrice - mean(test_data$SalePrice))^2)
r2_tree_complex <- 1 - sum((test_data$SalePrice - pred_complex_test)^2) / sum((test_data$SalePrice - mean(test_data$SalePrice))^2)
r2_tree_very_complex <- 1 - sum((test_data$SalePrice - pred_very_complex_test)^2) / sum((test_data$SalePrice - mean(test_data$SalePrice))^2)
r2_tree_extreme <- 1 - sum((test_data$SalePrice - pred_extreme_test)^2) / sum((test_data$SalePrice - mean(test_data$SalePrice))^2)
r2_tree_overfit <- 1 - sum((test_data$SalePrice - pred_overfit_test)^2) / sum((test_data$SalePrice - mean(test_data$SalePrice))^2)

# Create comparison data frame
tree_comparison <- data.frame(
  Model = c("Simple Tree", "Medium Tree", "Complex Tree", "Very Complex Tree", "Extreme Tree", "Overfit Tree"),
  Max_Depth = c(2, 4, 6, 8, 12, 20),
  RMSE_Test = c(rmse_tree_simple_test, rmse_tree_medium_test, rmse_tree_complex_test, rmse_tree_very_complex_test, rmse_tree_extreme_test, rmse_tree_overfit_test),
  RMSE_Train = c(rmse_tree_simple_train, rmse_tree_medium_train, rmse_tree_complex_train, rmse_tree_very_complex_train, rmse_tree_extreme_train, rmse_tree_overfit_train),
  R_squared = c(r2_tree_simple, r2_tree_medium, r2_tree_complex, r2_tree_very_complex, r2_tree_extreme, r2_tree_overfit)
)

print("Decision Tree Performance (showing overfitting):")

```

```
[1] "Decision Tree Performance (showing overfitting):"
```

```
print(tree_comparison)
```

	Model	Max_Depth	RMSE_Test	RMSE_Train	R_squared
1	Simple Tree	2	52470.52	45583.33	0.5671058
2	Medium Tree	4	40777.98	35065.39	0.7385417

3	Complex Tree	6	45370.93	34091.32	0.6763271
4	Very Complex Tree	8	41914.09	33232.10	0.7237699
5	Extreme Tree	12	44228.34	31957.86	0.6924241
6	Overfit Tree	20	44228.34	31957.86	0.6924241

Python

```
# Build decision trees with increasing complexity (with corrected categorical zipCode)
from sklearn.tree import DecisionTreeRegressor

tree_simple = DecisionTreeRegressor(max_depth=2, min_samples_split=20, min_samples_leaf=10,
tree_medium = DecisionTreeRegressor(max_depth=4, min_samples_split=20, min_samples_leaf=10,
tree_complex = DecisionTreeRegressor(max_depth=6, min_samples_split=15, min_samples_leaf=8,
tree_very_complex = DecisionTreeRegressor(max_depth=8, min_samples_split=10, min_samples_leaf=5,
tree_extreme = DecisionTreeRegressor(max_depth=12, min_samples_split=5, min_samples_leaf=2,
tree_overfit = DecisionTreeRegressor(max_depth=20, min_samples_split=2, min_samples_leaf=1,

# Fit models
tree_simple.fit(X_train, y_train)
```

```
DecisionTreeRegressor(max_depth=2, min_samples_leaf=10, min_samples_split=20,
                      random_state=123)
```

```
tree_medium.fit(X_train, y_train)
```

```
DecisionTreeRegressor(max_depth=4, min_samples_leaf=10, min_samples_split=20,
                      random_state=123)
```

```
tree_complex.fit(X_train, y_train)
```

```
DecisionTreeRegressor(max_depth=6, min_samples_leaf=8, min_samples_split=15,
                      random_state=123)
```

```
tree_very_complex.fit(X_train, y_train)
```

```
DecisionTreeRegressor(max_depth=8, min_samples_leaf=5, min_samples_split=10,
                      random_state=123)
```

```
tree_extreme.fit(X_train, y_train)
```

```
DecisionTreeRegressor(max_depth=12, min_samples_leaf=2, min_samples_split=5,  
                      random_state=123)
```

```
tree_overfit.fit(X_train, y_train)
```

```
DecisionTreeRegressor(max_depth=20, random_state=123)
```

```
# Calculate predictions for test data  
pred_simple_test = tree_simple.predict(X_test)  
pred_medium_test = tree_medium.predict(X_test)  
pred_complex_test = tree_complex.predict(X_test)  
pred_very_complex_test = tree_very_complex.predict(X_test)  
pred_extreme_test = tree_extreme.predict(X_test)  
pred_overfit_test = tree_overfit.predict(X_test)  
  
# Calculate predictions for training data  
pred_simple_train = tree_simple.predict(X_train)  
pred_medium_train = tree_medium.predict(X_train)  
pred_complex_train = tree_complex.predict(X_train)  
pred_very_complex_train = tree_very_complex.predict(X_train)  
pred_extreme_train = tree_extreme.predict(X_train)  
pred_overfit_train = tree_overfit.predict(X_train)  
  
# Calculate performance metrics for test data  
rmse_tree_simple_test = np.sqrt(mean_squared_error(y_test, pred_simple_test))  
rmse_tree_medium_test = np.sqrt(mean_squared_error(y_test, pred_medium_test))  
rmse_tree_complex_test = np.sqrt(mean_squared_error(y_test, pred_complex_test))  
rmse_tree_very_complex_test = np.sqrt(mean_squared_error(y_test, pred_very_complex_test))  
rmse_tree_extreme_test = np.sqrt(mean_squared_error(y_test, pred_extreme_test))  
rmse_tree_overfit_test = np.sqrt(mean_squared_error(y_test, pred_overfit_test))  
  
# Calculate performance metrics for training data  
rmse_tree_simple_train = np.sqrt(mean_squared_error(y_train, pred_simple_train))  
rmse_tree_medium_train = np.sqrt(mean_squared_error(y_train, pred_medium_train))  
rmse_tree_complex_train = np.sqrt(mean_squared_error(y_train, pred_complex_train))  
rmse_tree_very_complex_train = np.sqrt(mean_squared_error(y_train, pred_very_complex_train))  
rmse_tree_extreme_train = np.sqrt(mean_squared_error(y_train, pred_extreme_train))  
rmse_tree_overfit_train = np.sqrt(mean_squared_error(y_train, pred_overfit_train))
```

```

r2_tree_simple = r2_score(y_test, pred_simple_test)
r2_tree_medium = r2_score(y_test, pred_medium_test)
r2_tree_complex = r2_score(y_test, pred_complex_test)
r2_tree_very_complex = r2_score(y_test, pred_very_complex_test)
r2_tree_extreme = r2_score(y_test, pred_extreme_test)
r2_tree_overfit = r2_score(y_test, pred_overfit_test)

# Create comparison data frame
tree_comparison_data = {
    'Model': ['Simple Tree', 'Medium Tree', 'Complex Tree', 'Very Complex Tree', 'Extreme Tree', 'Overfit Tree'],
    'Max_Depth': [2, 4, 6, 8, 12, 20],
    'RMSE_Test': [rmse_tree_simple_test, rmse_tree_medium_test, rmse_tree_complex_test, rmse_tree_very_complex_test, rmse_tree_extreme_test, rmse_tree_overfit_test],
    'RMSE_Train': [rmse_tree_simple_train, rmse_tree_medium_train, rmse_tree_complex_train, rmse_tree_very_complex_train, rmse_tree_extreme_train, rmse_tree_overfit_train],
    'R_squared': [r2_tree_simple, r2_tree_medium, r2_tree_complex, r2_tree_very_complex, r2_tree_extreme, r2_tree_overfit]
}

tree_comparison_df = pd.DataFrame(tree_comparison_data)
print("Decision Tree Performance (showing overfitting):")

```

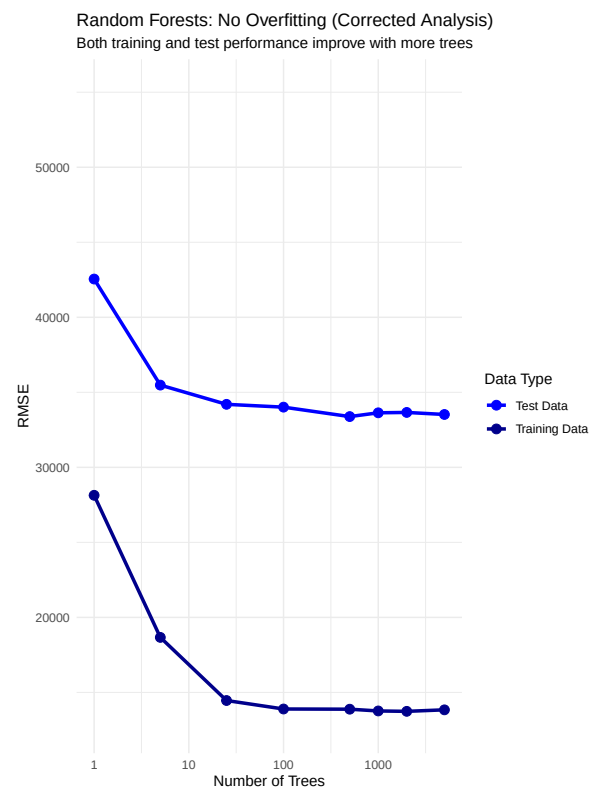
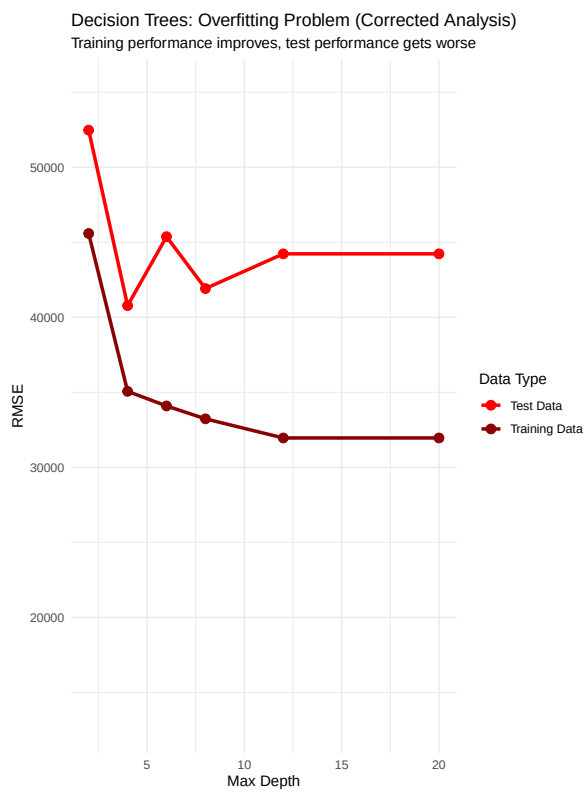
Decision Tree Performance (showing overfitting):

```
print(tree_comparison_df)
```

	Model	Max_Depth	RMSE_Test	RMSE_Train	R_squared
0	Simple Tree	2	46638.437196	47309.123100	0.522480
1	Medium Tree	4	41762.299650	37210.867627	0.617112
2	Complex Tree	6	36908.232312	30727.145719	0.700946
3	Very Complex Tree	8	38717.276480	24628.488220	0.670911
4	Extreme Tree	12	37856.829887	14914.680252	0.685376
5	Overfit Tree	20	41238.625967	729.695876	0.626654

The Overfitting Visualization

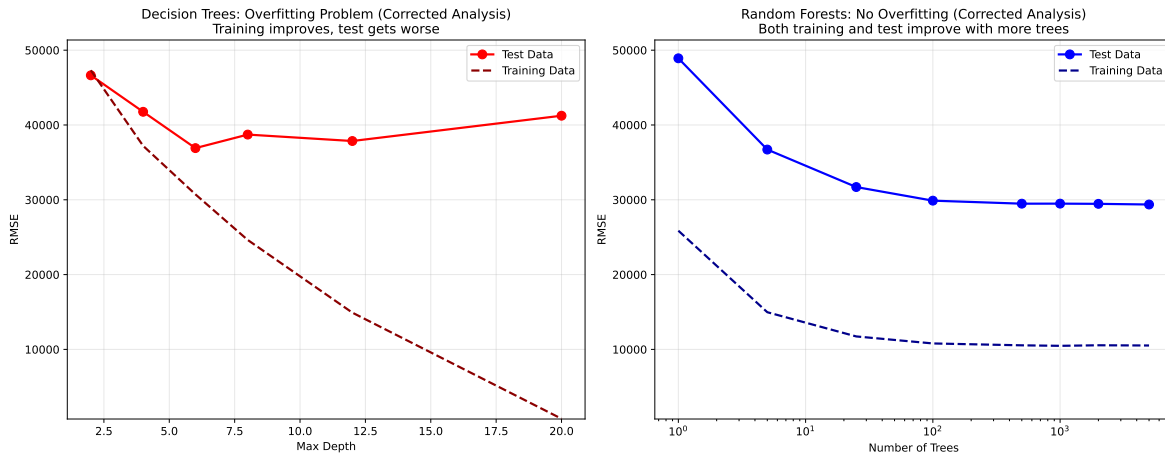
R



Python

(693.2110823469193, 51360.932518253845)

(693.2110823469193, 51360.932518253845)



The Key Insight: Why Random Forests Don't Overfit



The Overfitting Problem Revealed

Decision Trees: As we make individual trees more complex (deeper, more splits), they start to memorize the training data and perform worse on new data. This is classic overfitting.

Random Forests: Even with 10,000 trees, performance never gets worse. Why?

1. **Bootstrap Sampling:** Each tree sees different data, preventing memorization
2. **Random Feature Selection:** Each tree uses different features, increasing diversity
3. **Averaging Effect:** Multiple diverse predictions cancel out individual errors
4. **No Single Tree Dominates:** No one tree can overfit the entire dataset

The Magic: Random forests turn the overfitting problem into a strength by using many diverse, slightly overfitted trees that average out to a robust prediction.

Simple Demonstration: Why More Trees Help

Let's look at a simple example to understand why ensemble methods work so well.

The Wisdom of Crowds

Think About It

Imagine you're trying to predict the price of a house. You ask 5 real estate agents, and each gives you a different estimate:

- Agent 1: \$250,000
- Agent 2: \$280,000
- Agent 3: \$270,000
- Agent 4: \$260,000
- Agent 5: \$290,000

Average prediction: \$270,000

Now imagine you ask 100 agents instead. Even if some agents are way off, the average of 100 predictions will be much more accurate than any single prediction.

This is exactly how random forests work!

Key Takeaway

More trees = Better predictions (up to a point)

- 1 tree: Single decision tree (baseline)
- 5 trees: Basic ensemble, some improvement
- 25 trees: Good ensemble, noticeable improvement
- 100 trees: Strong ensemble, significant improvement
- 500 trees: Very strong ensemble, diminishing returns
- 1,000 trees: Excellent ensemble, minimal additional improvement
- 2,000 trees: Very strong ensemble, tiny additional improvement
- 5,000 trees: Maximum ensemble, virtually no additional improvement

The beauty of random forests is that **you can't overfit by adding more trees** - they only make your model better! Notice how the improvement becomes smaller as we add more trees, but performance never gets worse.

Discussion Questions for Challenge

1. **The Ensemble Effect:** Looking at the performance results, what pattern do you see as the number of trees increases? Why do you think more trees lead to better performance? What would happen if we used 10,000 trees instead of 5,000?

2. **No Overfitting Mystery:** Unlike other machine learning algorithms, random forests don't overfit when you add more trees. Why do you think this is the case? What mechanisms in random forests prevent overfitting even with many trees?
3. **Practical Implications:** If you were building a real estate price prediction model for a business, how many trees would you choose? Consider both accuracy and computational cost. What factors would influence your decision?

! Important Note on AI Usage

No coding assistance expected: This challenge is designed for independent analysis and critical thinking. You are **not** expected to use AI for coding help or to write code for you. You are **not** expected to code at all unless curious about any ideas you may have.

AI as unreliable thought partner: If you choose to use AI tools, treat them as an unreliable thought partner. AI responses should be verified and cross-checked rather than accepted at face value.

Focus on understanding: The key learning objective is understanding why ensemble methods work and how random forests avoid overfitting. Focus on the conceptual understanding rather than technical implementation details.